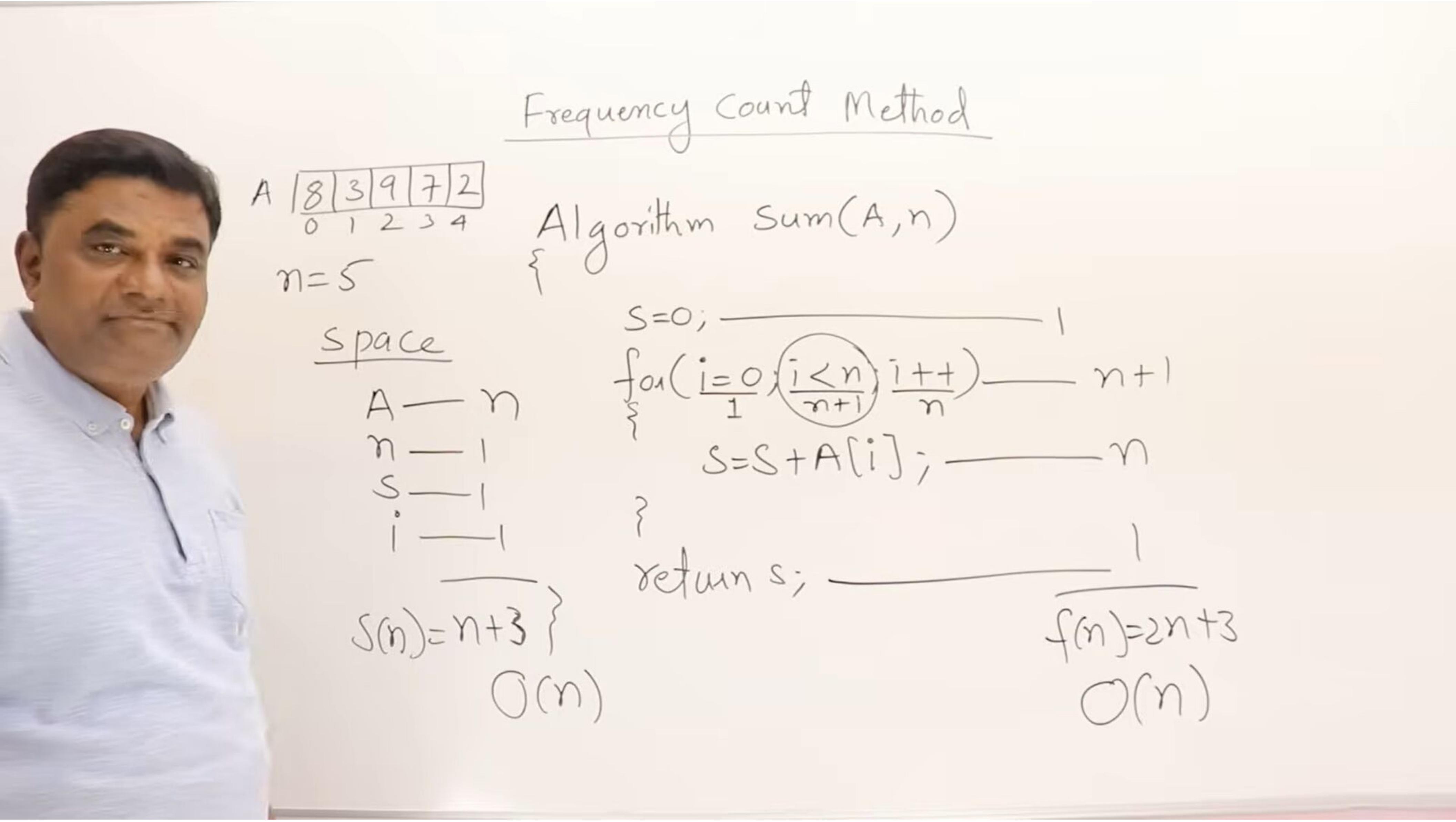
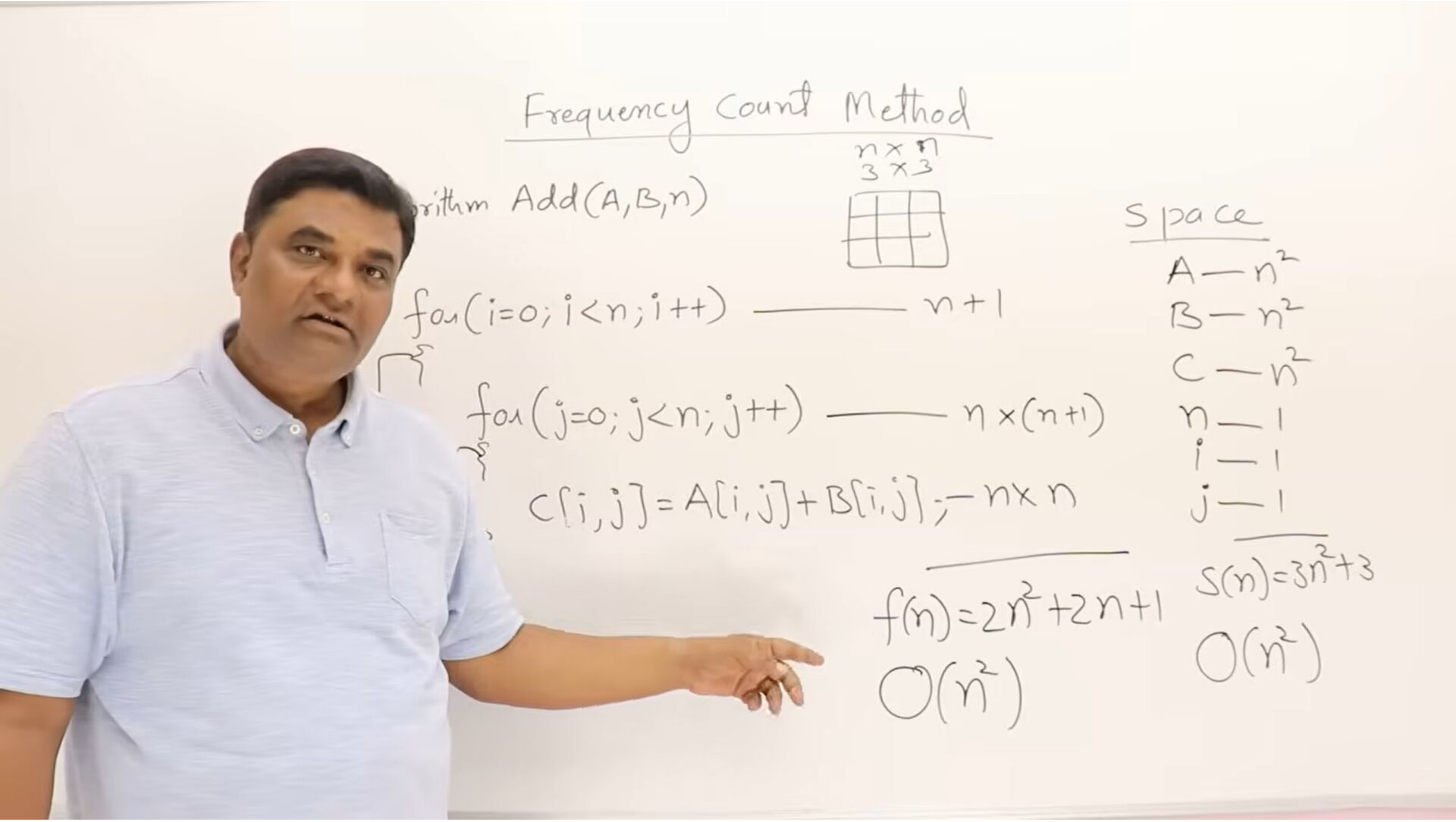

Frequency Count Method
A | 8 | 3 | 9 | 7 | 2 |
0 1 2 3 4
n = 5
Algorithm Sum(A, n)
{
S=0; —— 1
for(i=0 ; i<n ; i++) —— n+1
1 n+1 n
{
S=S+A[i]; —— n
}
return s; —— 1
f(n)=2n+3
O(n)
Space
A — n
n — 1
S — 1
i — 1
S(n)=n+3
O(n)



Frequency Count Method
n x n
3 x 3
orithm Add(A,B,n)
Space
A — n^2
B — n^2
C — n^2
n — 1
i — 1
j — 1
for(i=0; i<n; i++) ——— n+1
for(j=0; j<n; j++) ——— n x (n+1)
C[i,j]=A[i,j]+B[i,j]; — n x n
f(n)=2n^2+2n+1
O(n^2)
S(n)=3n^2+3
O(n^2)

Frequency Count Method

```
          Algorithm Multiply(A,B,n)
          {
n+1         ———— for(i=0; i<n; i++)
                 {
(n+1) n     ———— for(j=0; j<n; j++)
                 {
n×n         ———— C[i,j]=0;
(n+1)×n×n   ———— for(k=0; k<n; k++)
                 {
n×n×n       ———— C[i,j]=C[i,j]+A[i,k]*B[k,j];
                 }
                 }
                 }
          }
```

$f(n)=2n^3+3n^2+2n+1$

$O(n^3)$

Time — $O(n^3)$

Space — $O(n^2)$

Space

A — n^2

B — n^2

C — n^2

n — 1

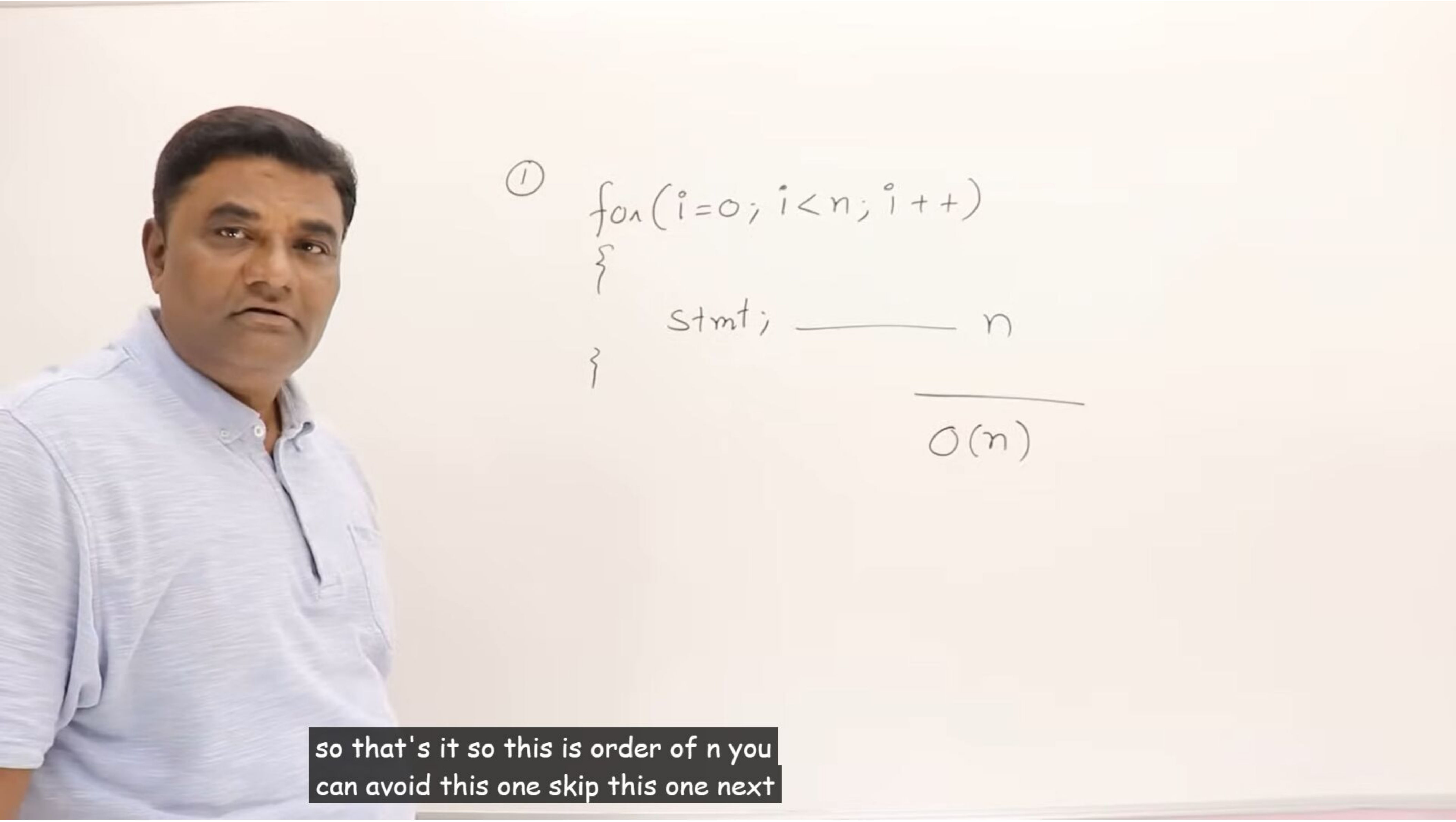
i — 1

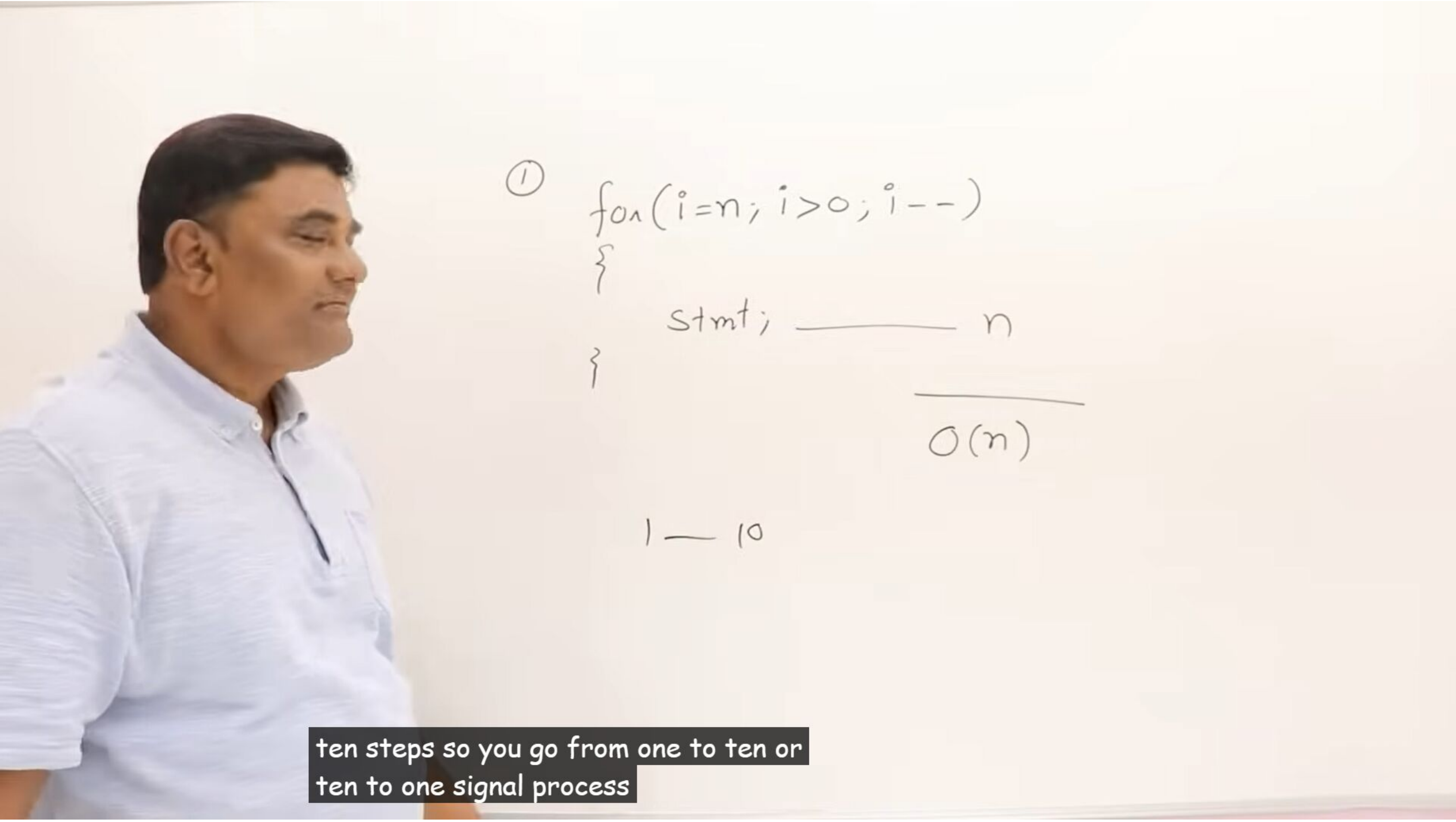
j — 1

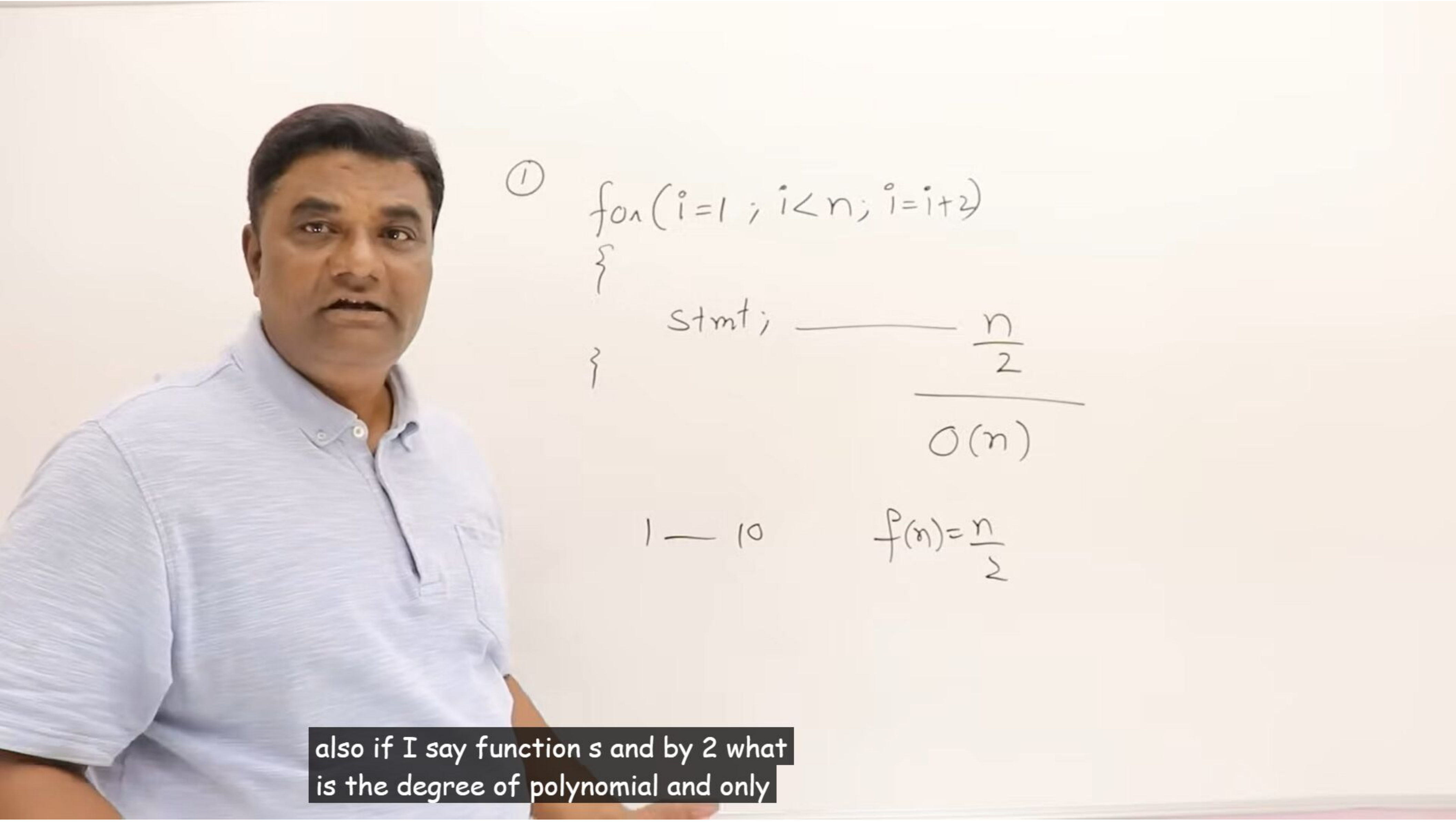
k — 1

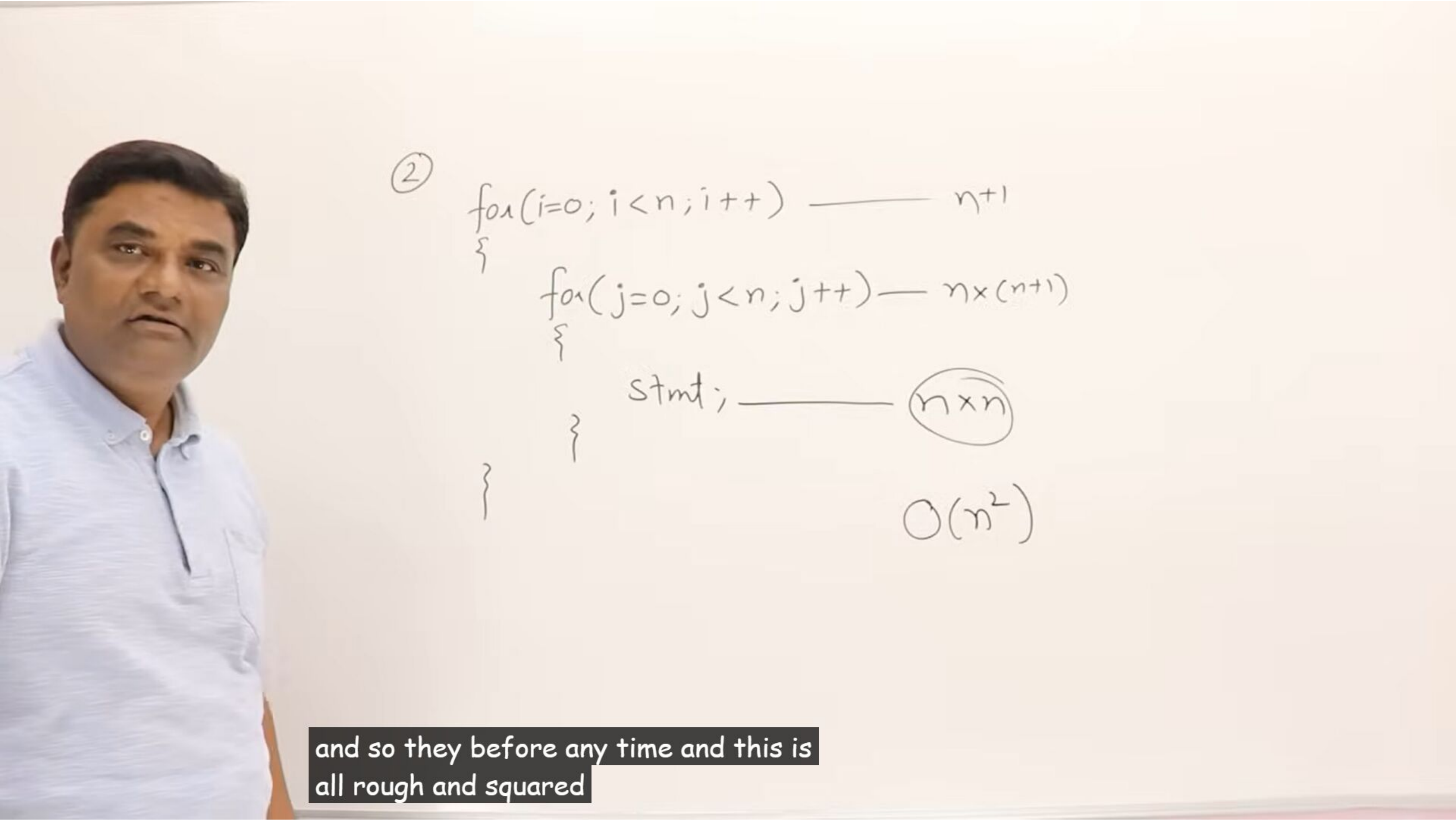
$S(n)=3n^2+4$

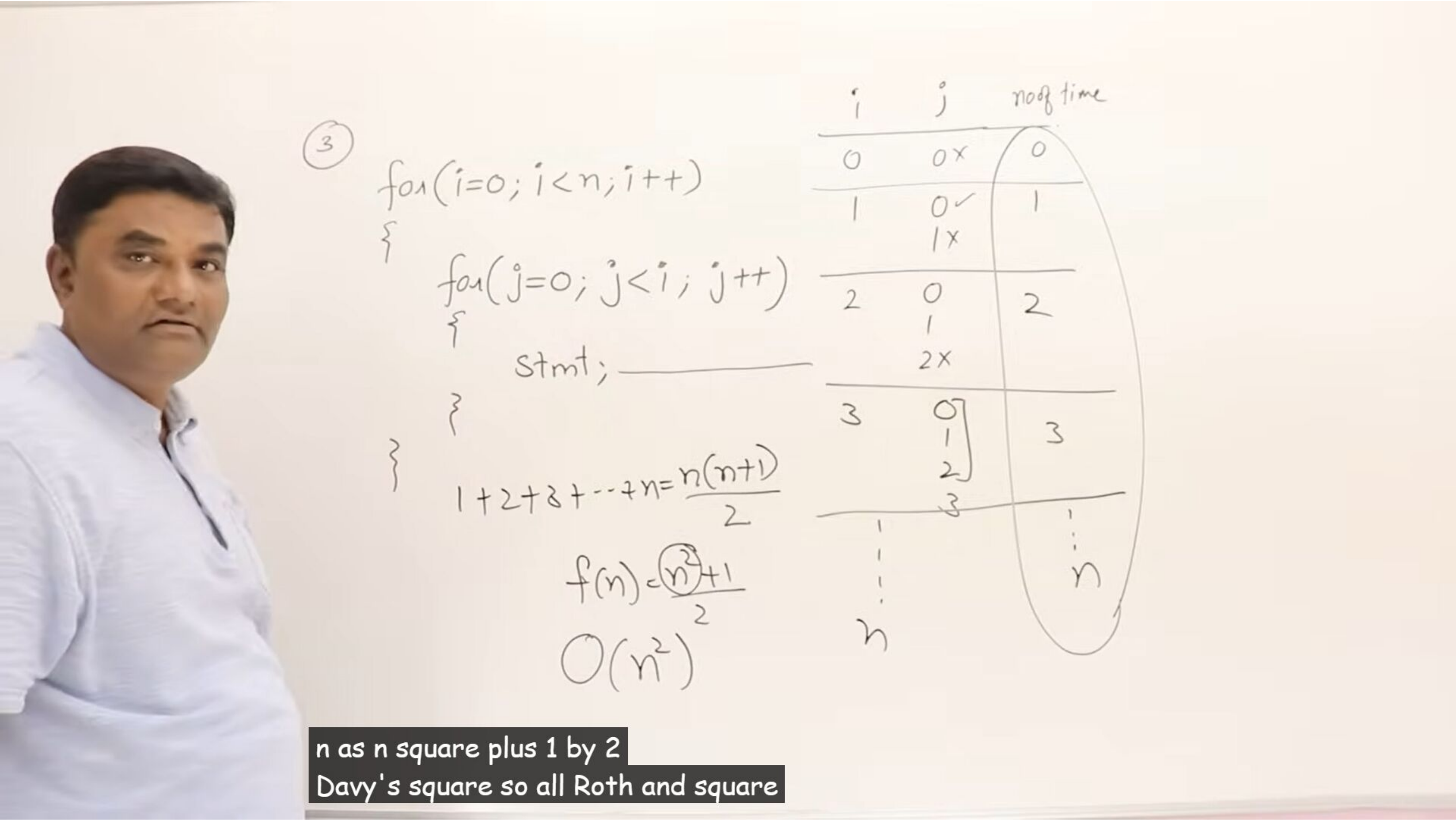
$O(n^2)$


1
for(i=0; i<n; i++)
{
stmt; n
}
O(n)
so that's it so this is order of n you
can avoid this one skip this one next


①
for(i=n; i>0; i--)
{
stmt; ——— n
}
O(n)
1 — 10
ten steps so you go from one to ten or
ten to one signal process


1
for(i=1 ; i<n; i=i+2)
{
stmt; n/2
}
O(n)
1 — 10
f(n)= n/2
also if I say function s and by 2 what
is the degree of polynomial and only


②
for(i=0; i<n; i++) —— n+1
{
for(j=0; j<n; j++) — n×(n+1)
{
stmt; —— n×n
}
}
O(n²)
and so they before any time and this is
all rough and squared



③
for(i=0; i<n; i++)
{
for(j=0; j<i; j++)
{
stmt;
}
}
1+2+3+...+n = n(n+1)/2
f(n) = (n²+1)/2
O(n²)
i
j
no of time
0
0 x
0
1
0 ✓
1 x
1
2
0
1
2 x
2
3
0
1
2
3
3
n
n
n as n square plus 1 by 2
Davy's square so all Roth and square

(4)

```
p=0;
for(i=1; p<=n; i++)
{
    p=p+i;
}
```

i	p
1	0+1=1
2	1+2=3
3	1+2+3
4	1+2+3+4
⋮	
k	1+2+3+4+...+k

Assume $p > n$

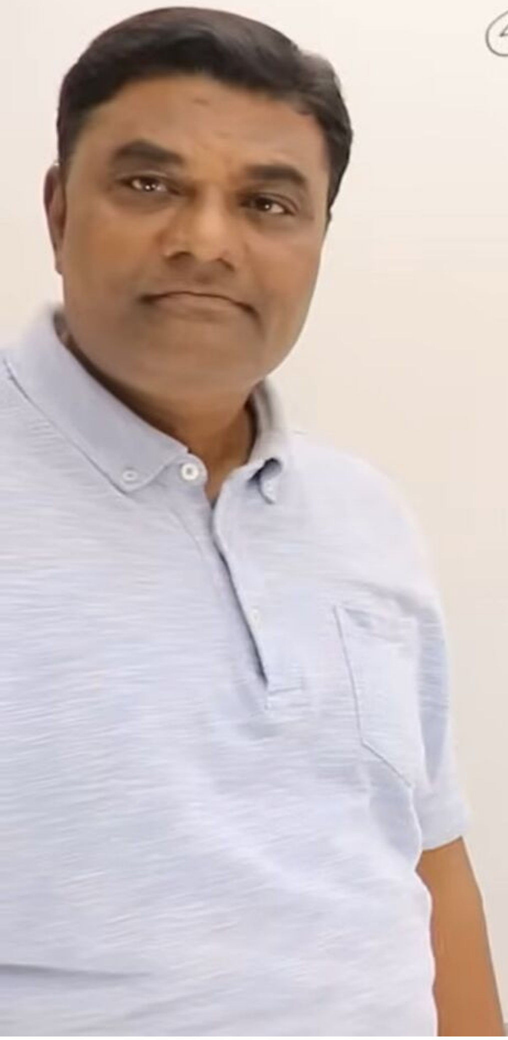
$$\because p = \frac{k(k+1)}{2}$$

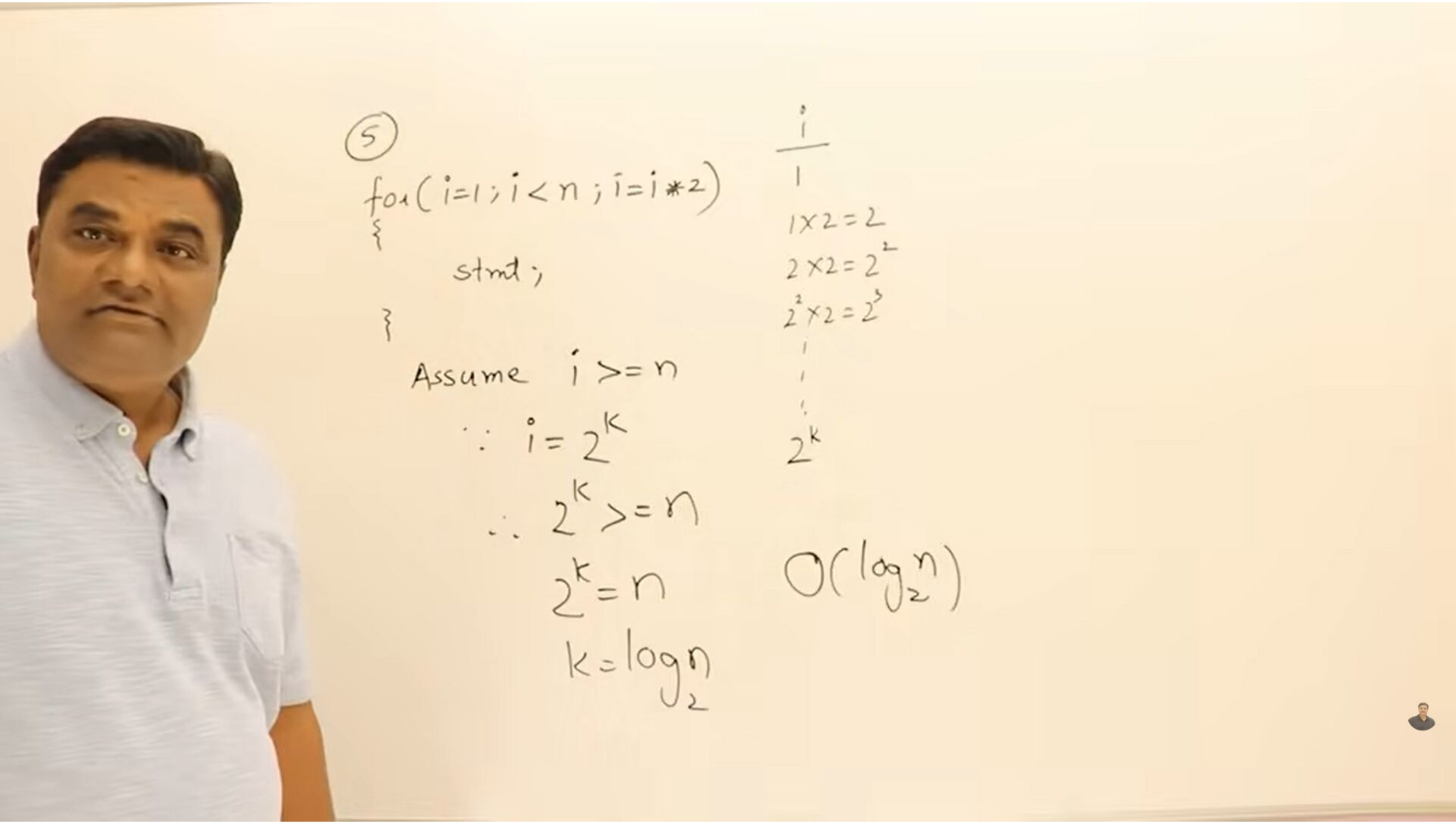
$$\frac{k(k+1)}{2} > n$$

$$k^2 > n$$

$$k > \sqrt{n}$$

$$O(\sqrt{n})$$





5
for(i=1; i<n; i=i*2)
{
stmt;
}
Assume $i >= n$
$\because i = 2^k$
$\therefore 2^k >= n$
$2^k = n$
$k = \log_2 n$
i
1
$1 \times 2 = 2$
$2 \times 2 = 2^2$
$2^2 \times 2 = 2^3$
2^k
$O(\log_2 n)$

5

```
for(i=1; i<n; i=i*2)
{
    stmt;
}
```

$i = 1 \times 2 \times 2 \times 2 - \cdots = n$

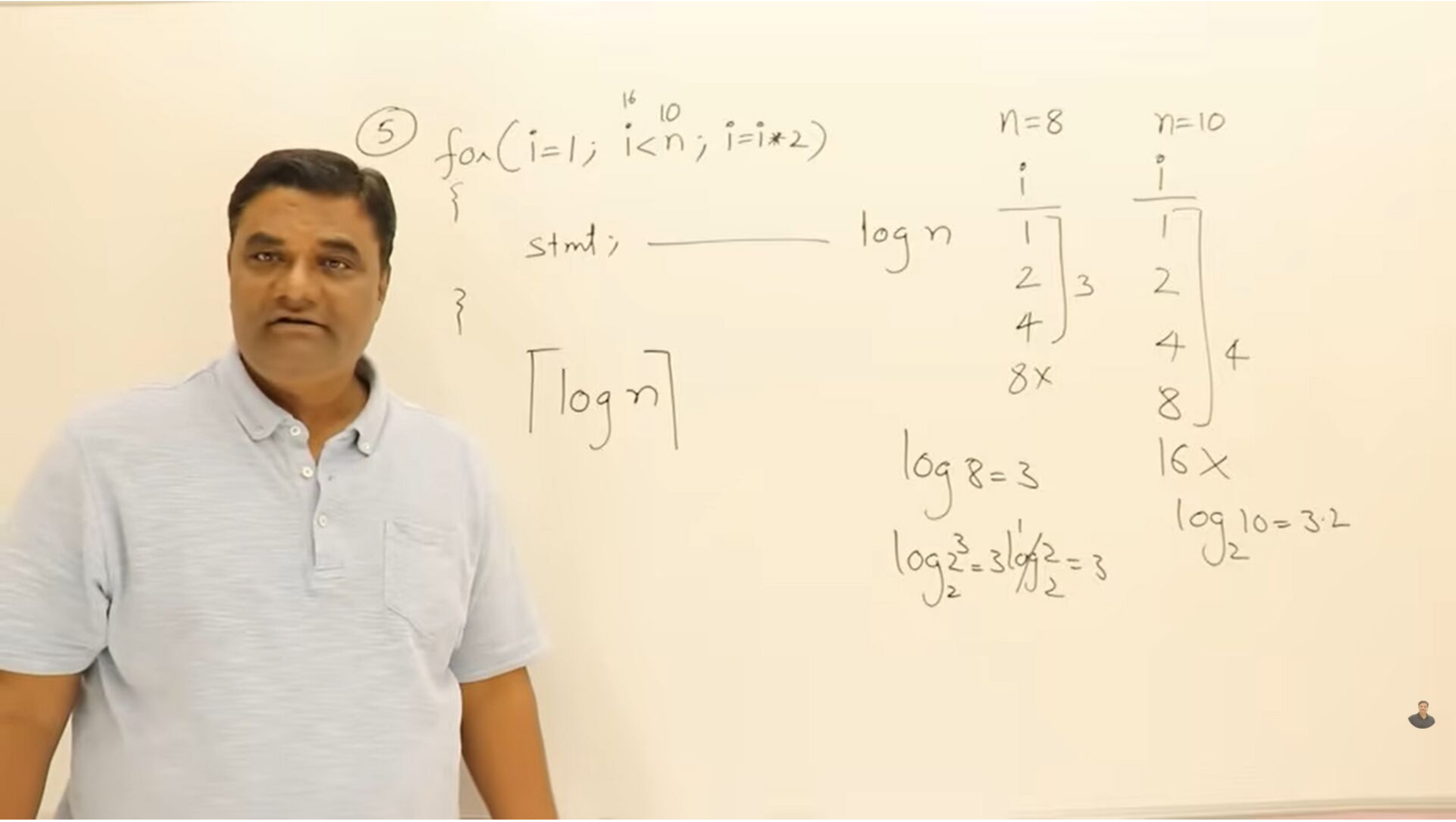
$2^k = n$

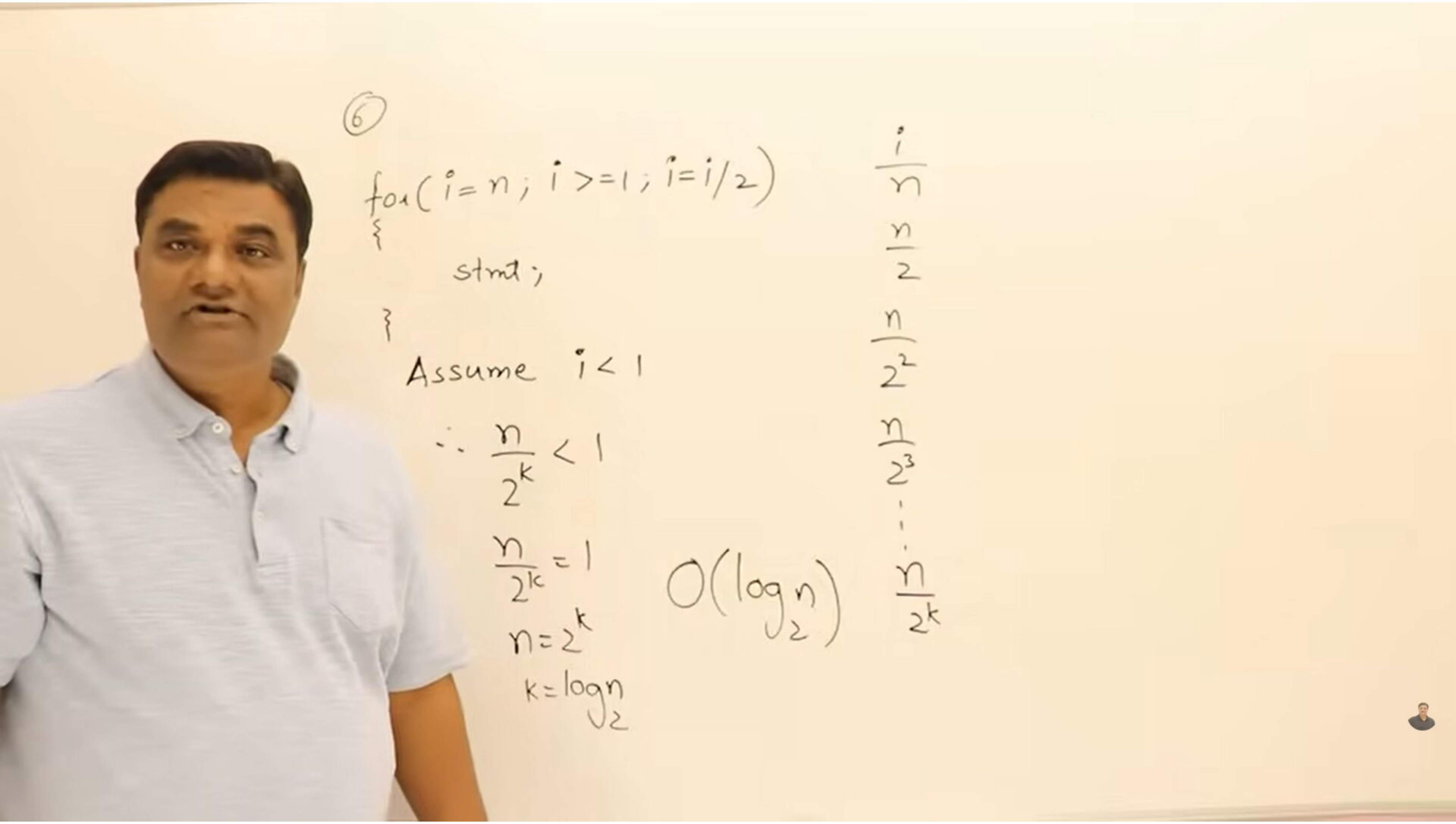
$k = \log_2 n$

```
for(i=1; i<=n; i++)
{
    stmt;
}
```

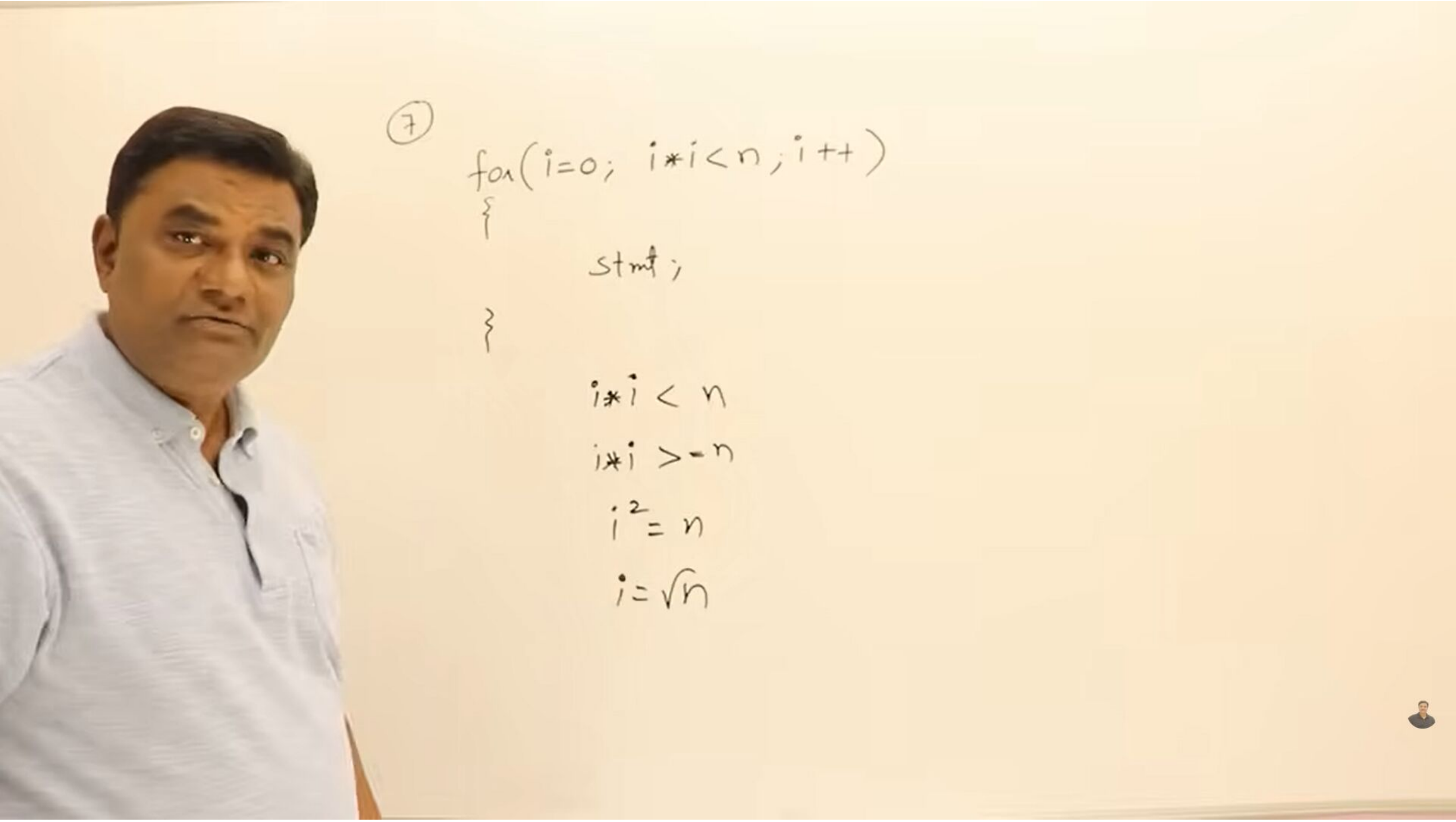
$i = \underbrace{1 + 1 + 1 + 1 \cdots + 1}_{} = n$

$k = n$


5
16 10
for(i=1; i<n; i=i*2)
{
stmt; —— log n
}
⌈log n⌉
n=8
i
1
2
4
3
8x
n=10
i
1
2
4
8
4
16x
log 8=3
log_2 2^3 = 3 log_2 2 = 3
1
log_2 10 = 3.2



6
for(i=n; i>=1; i=i/2)
{
stmt;
}
Assume $i < 1$
$\therefore \frac{n}{2^k} < 1$
$\frac{n}{2^k} = 1$
$n = 2^k$
$k = \log_2 n$
$O(\log_2 n)$
i
n
$\frac{n}{2}$
$\frac{n}{2^2}$
$\frac{n}{2^3}$
$\frac{n}{2^k}$


7
for(i=0; i*i<n; i++)
{
stmt;
}
i*i < n
i*i >= n
i^2 = n
i = √n

(8)

```
for(i=0; i<n; i++)
{
    stmt;  ———————— n
}
for(j=0; j<n; j++)
{
    stmt;  ———————— n
}
```

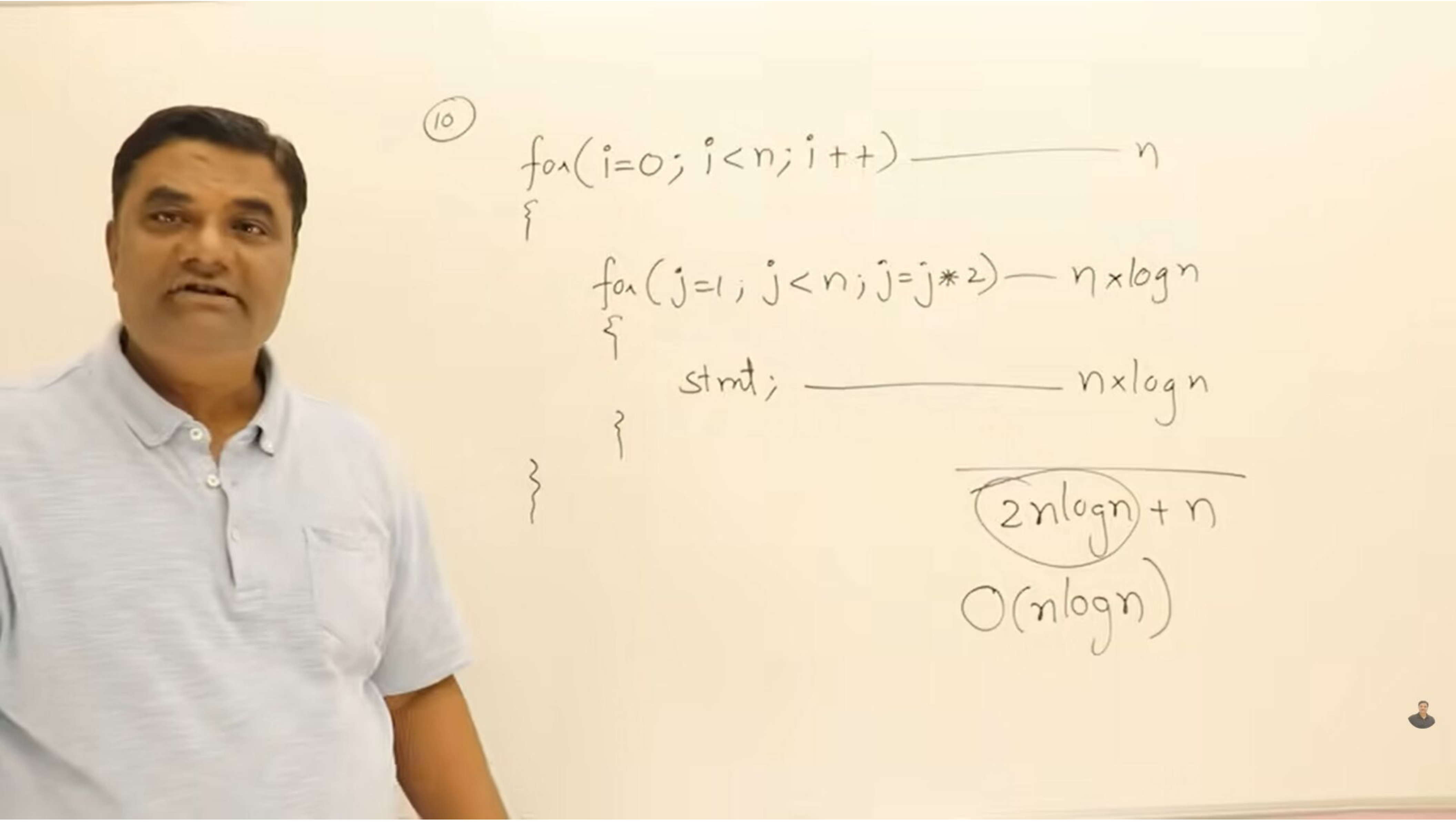
$$2n$$

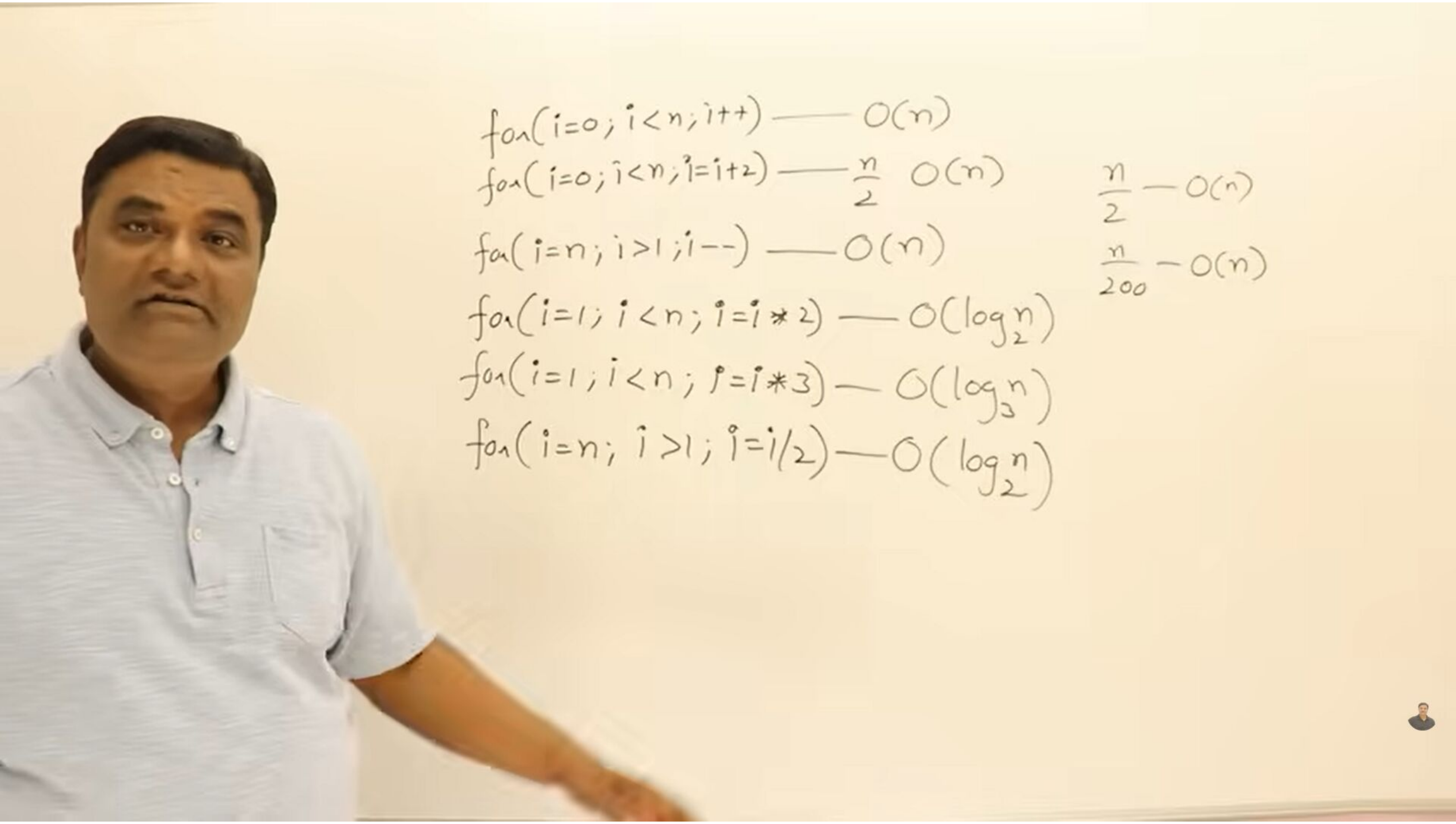
$$O(n)$$

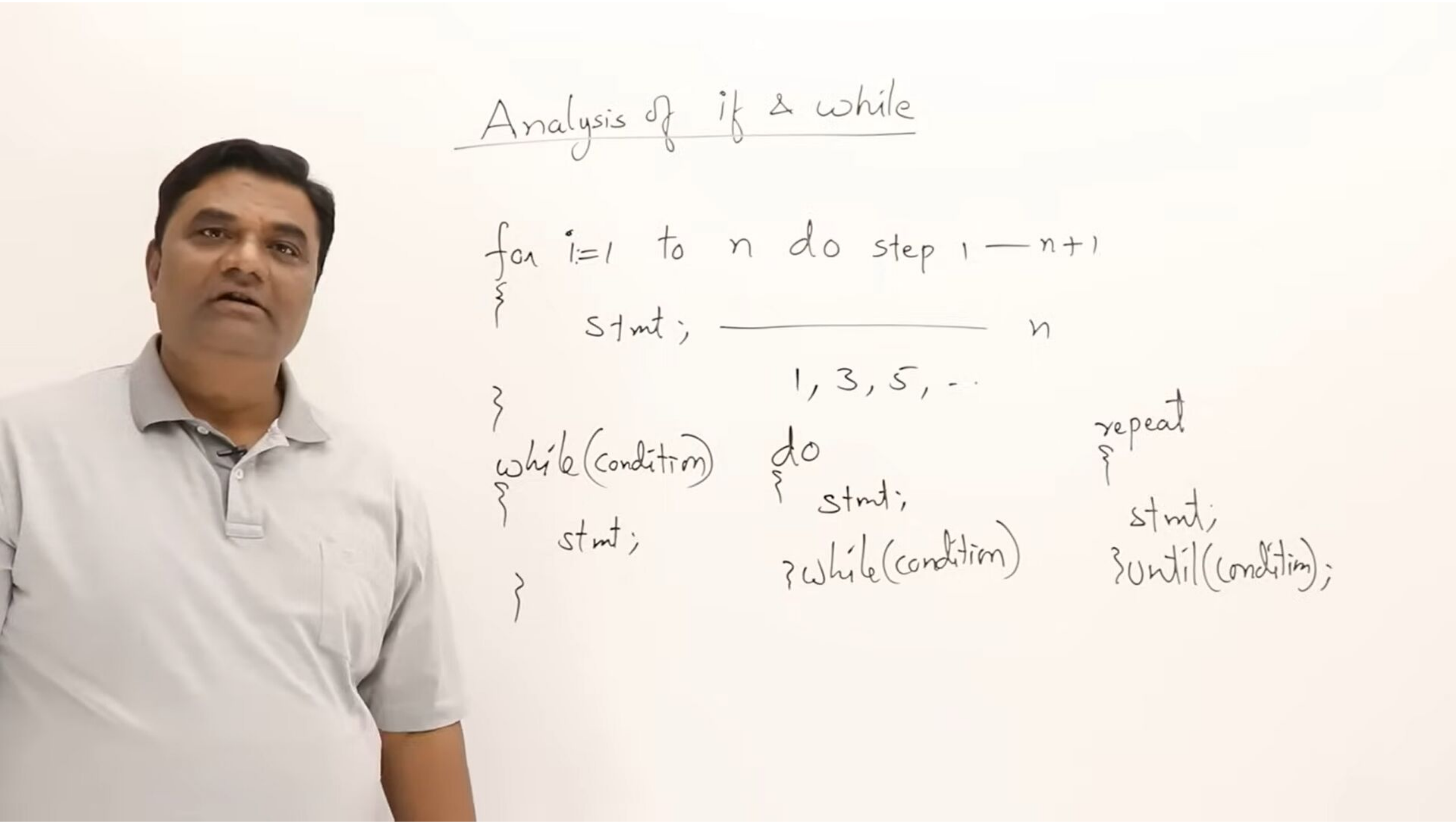
9 p=0

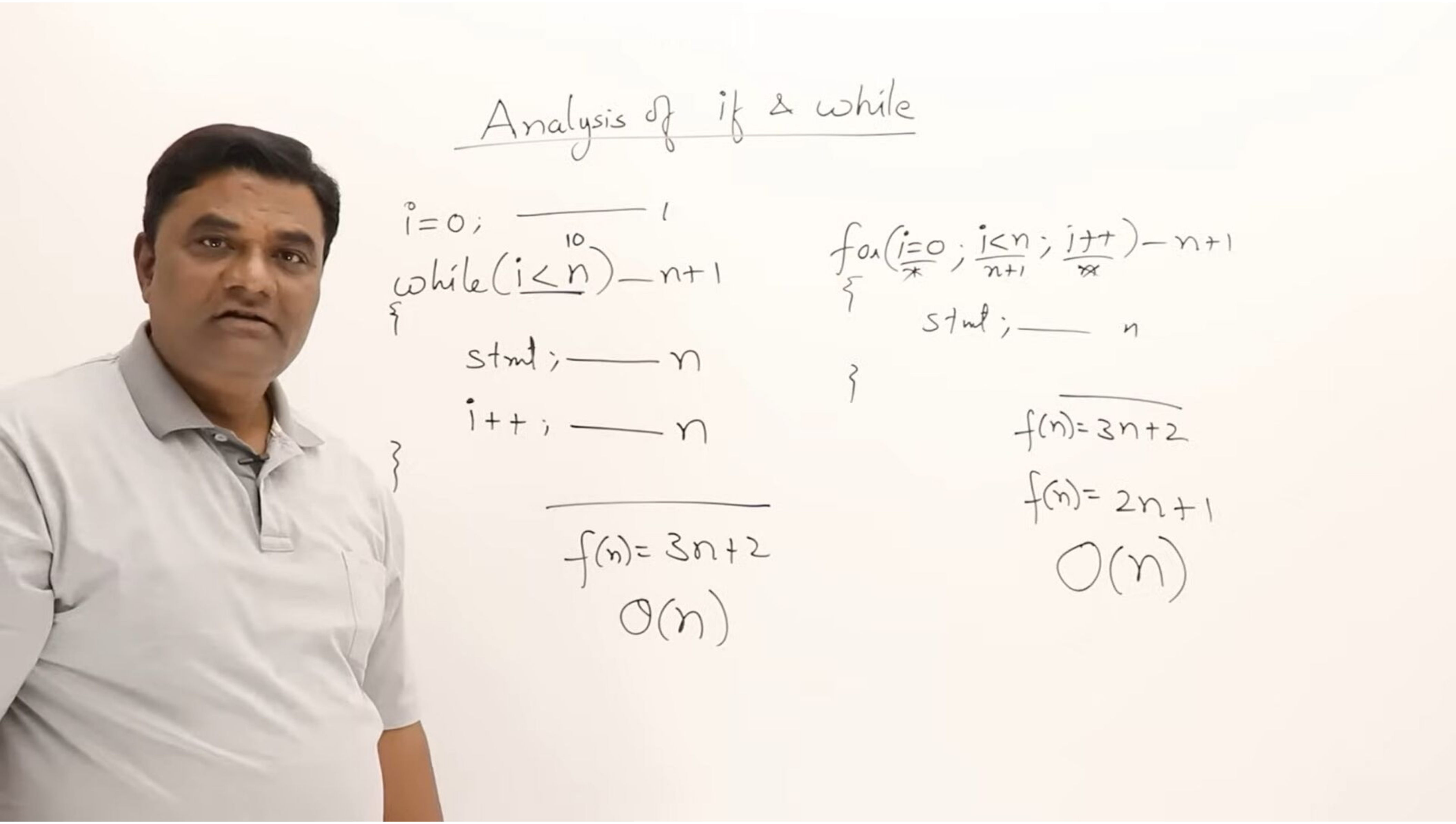
```
for(i=1; i<n; i=i*2)
{
    p++;        ----------  p=log n
}
for(j=1; j<p; j=j*2)
{
    stmt;       ----------  log p
}
```

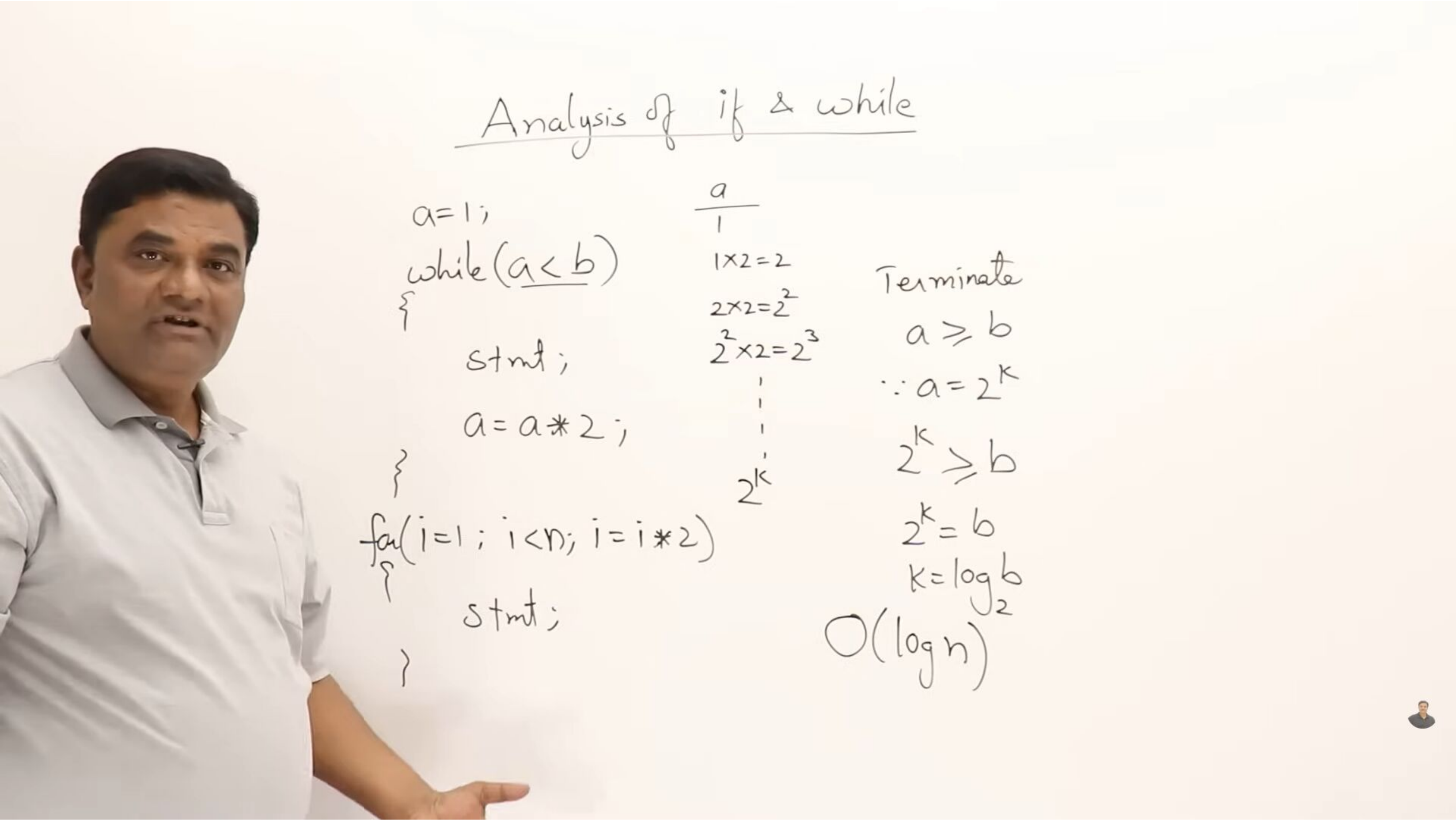
$O(\log \log n)$

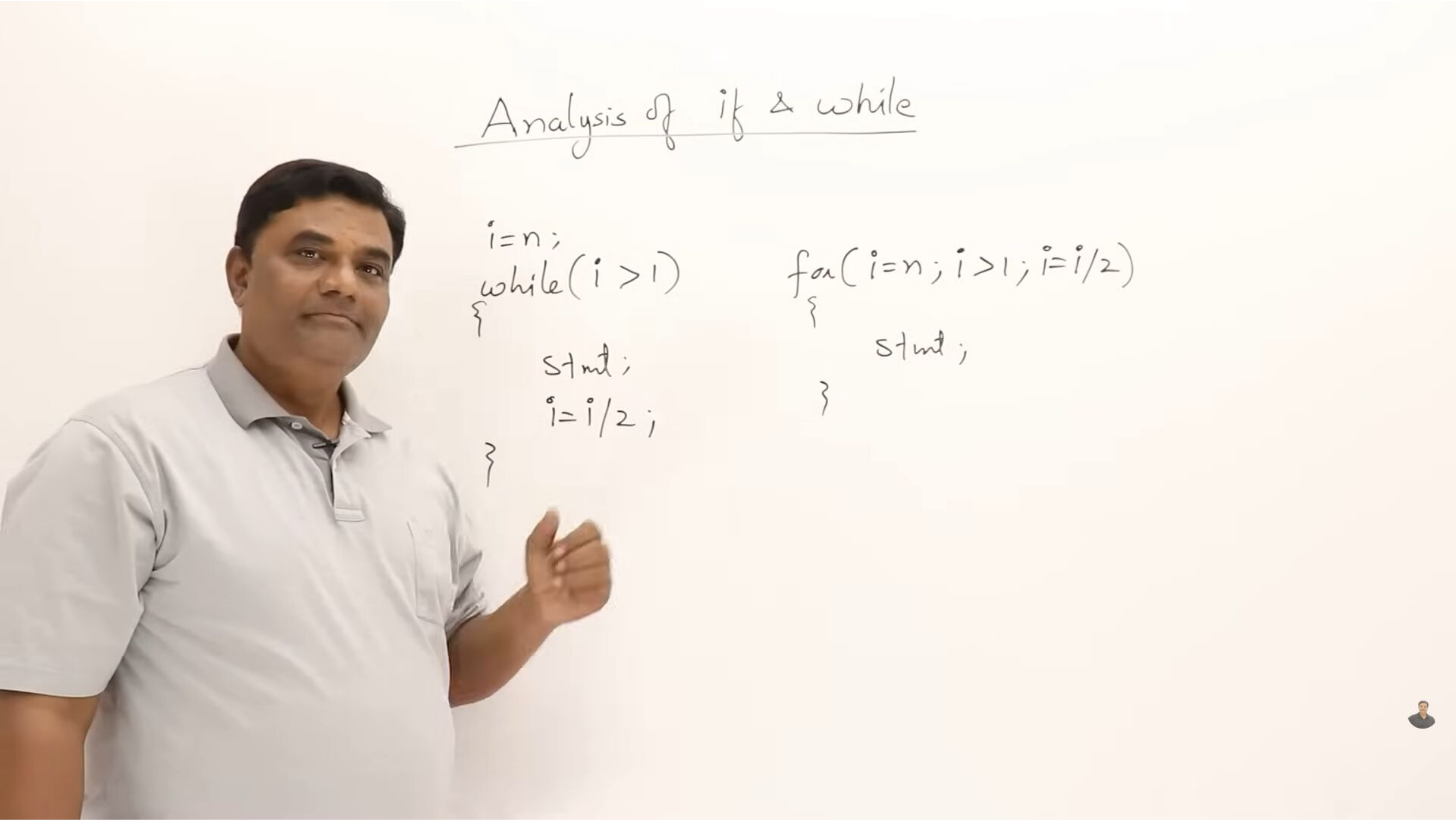

10
for(i=0; i<n; i++) n
{
for(j=1; j<n; j=j*2) n×logn
{
stmt; n×logn
}
}
2nlogn + n
O(nlogn)

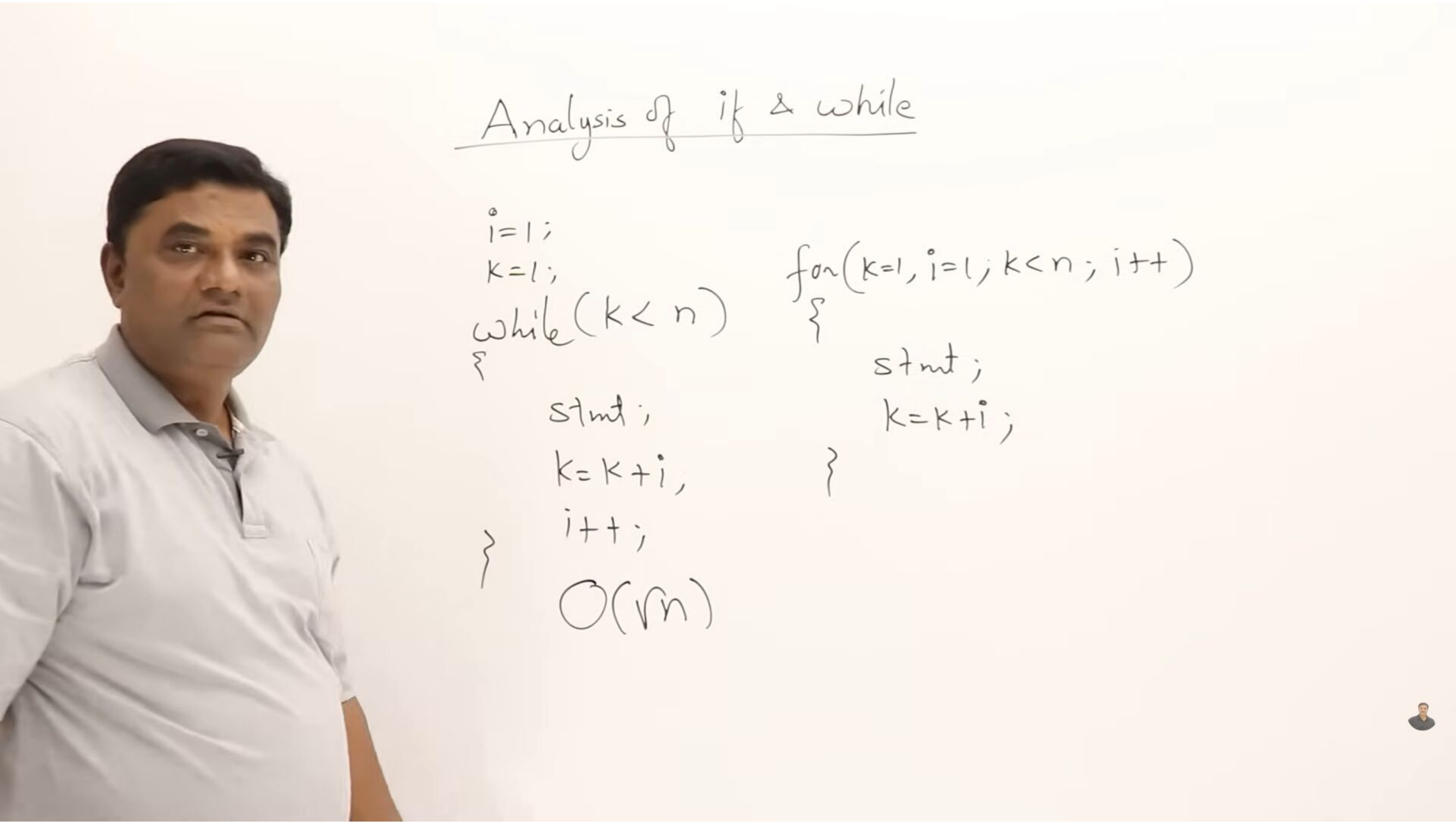

for(i=0; i<n; i++) —— O(n)
for(i=0; i<n; i=i+2) —— n/2 O(n)
for(i=n; i>1; i--) —— O(n)
for(i=1; i<n; i=i*2) —— O(log₂ n)
for(i=1; i<n; i=i*3) — O(log₃ n)
for(i=n; i>1; i=i/2) — O(log₂ n)
n/2 — O(n)
n/200 — O(n)

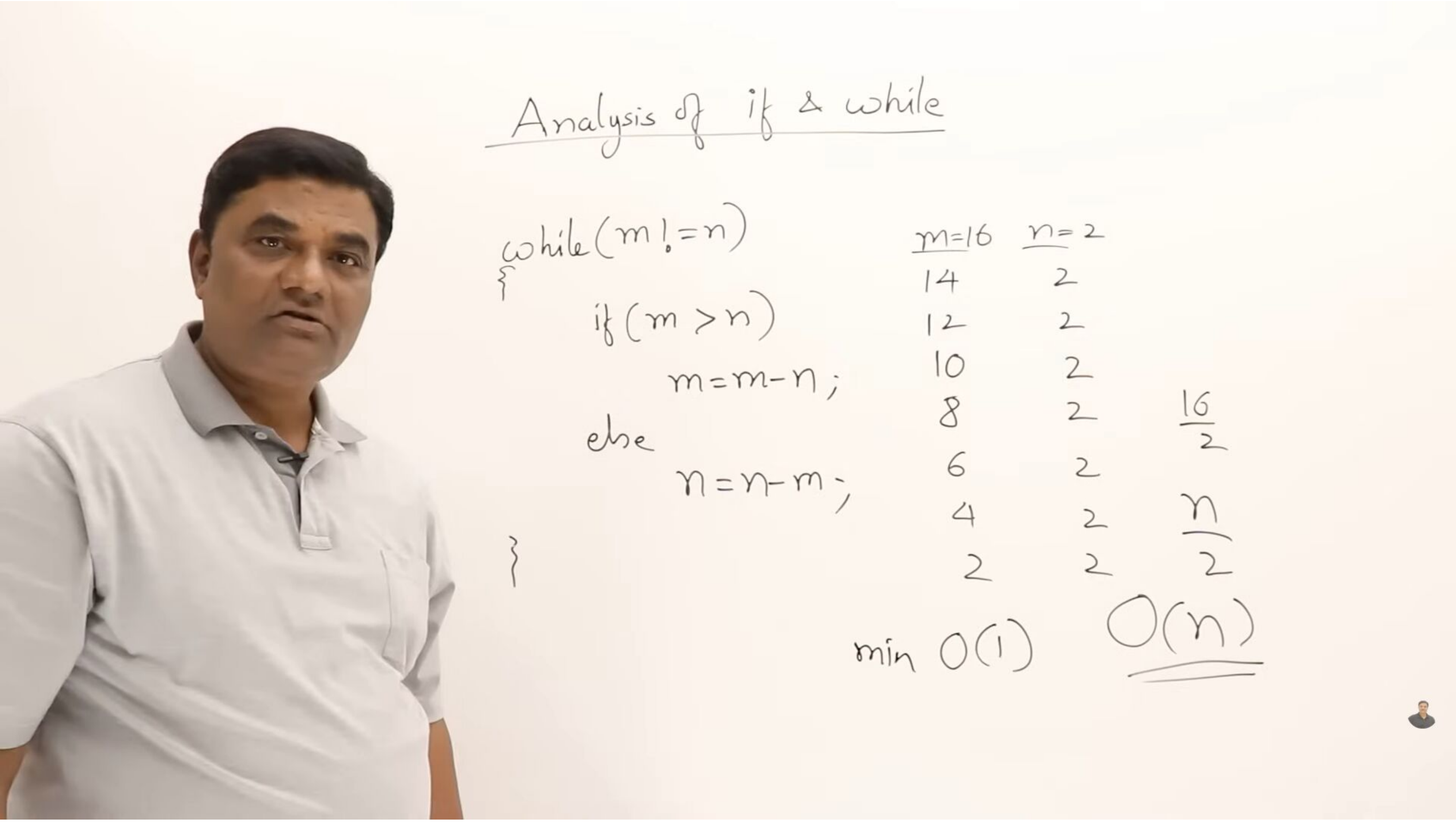

Analysis of if & while
for i:=1 to n do step 1 — n+1
{
stmt; — n
1, 3, 5, ...
}
while(condition)
{
stmt;
}
do
{
stmt;
}while(condition)
repeat
{
stmt;
}until(condition);

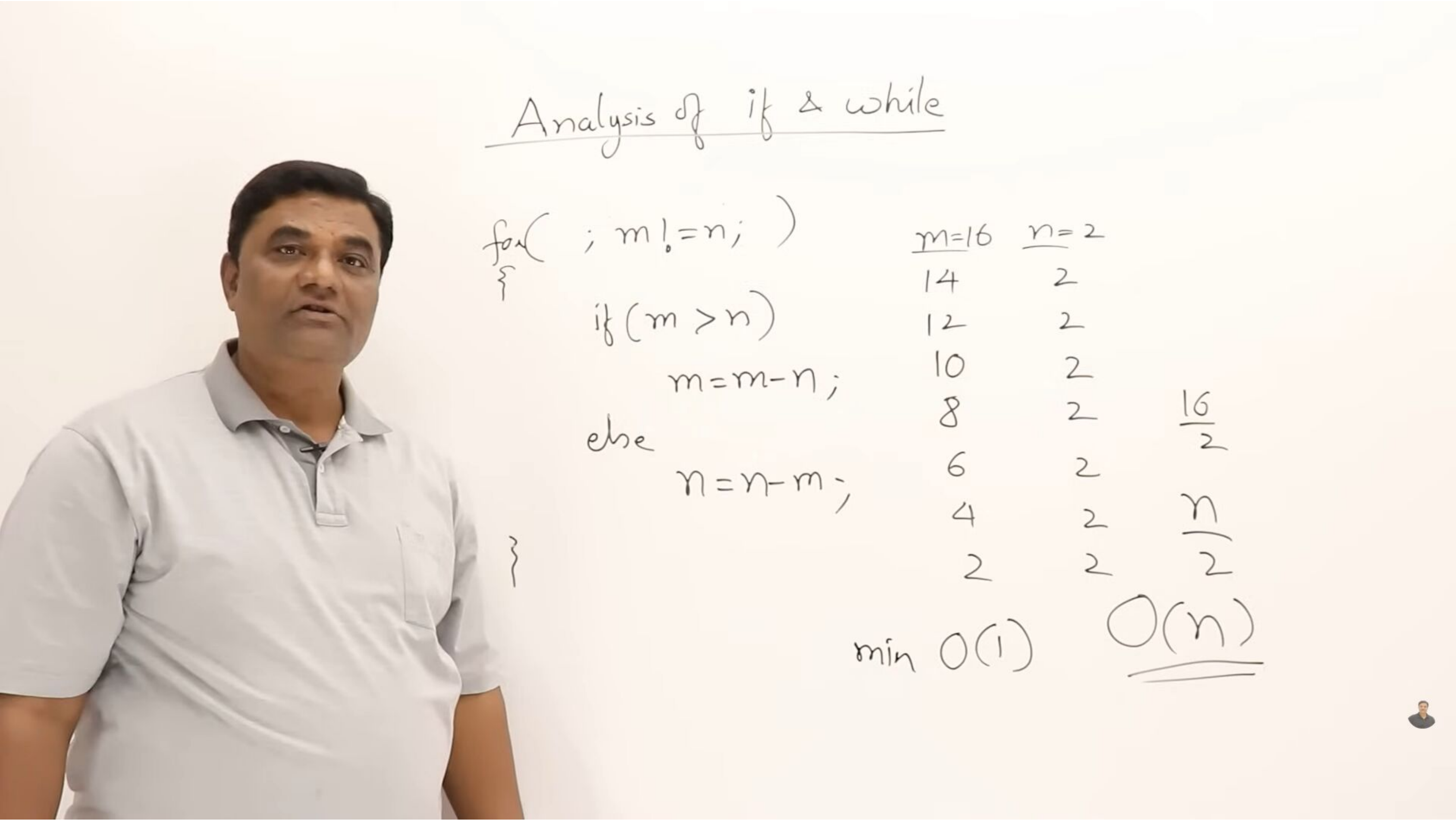

Analysis of if & while
i=0; — 1
10
while(i<n) — n+1
{
stmt; — n
i++; — n
}
f(n)= 3n+2
O(n)
for(i=0; i<n; i++) — n+1
n+1
{
stmt; — n
}
f(n)= 3n+2
f(n)= 2n+1
O(n)

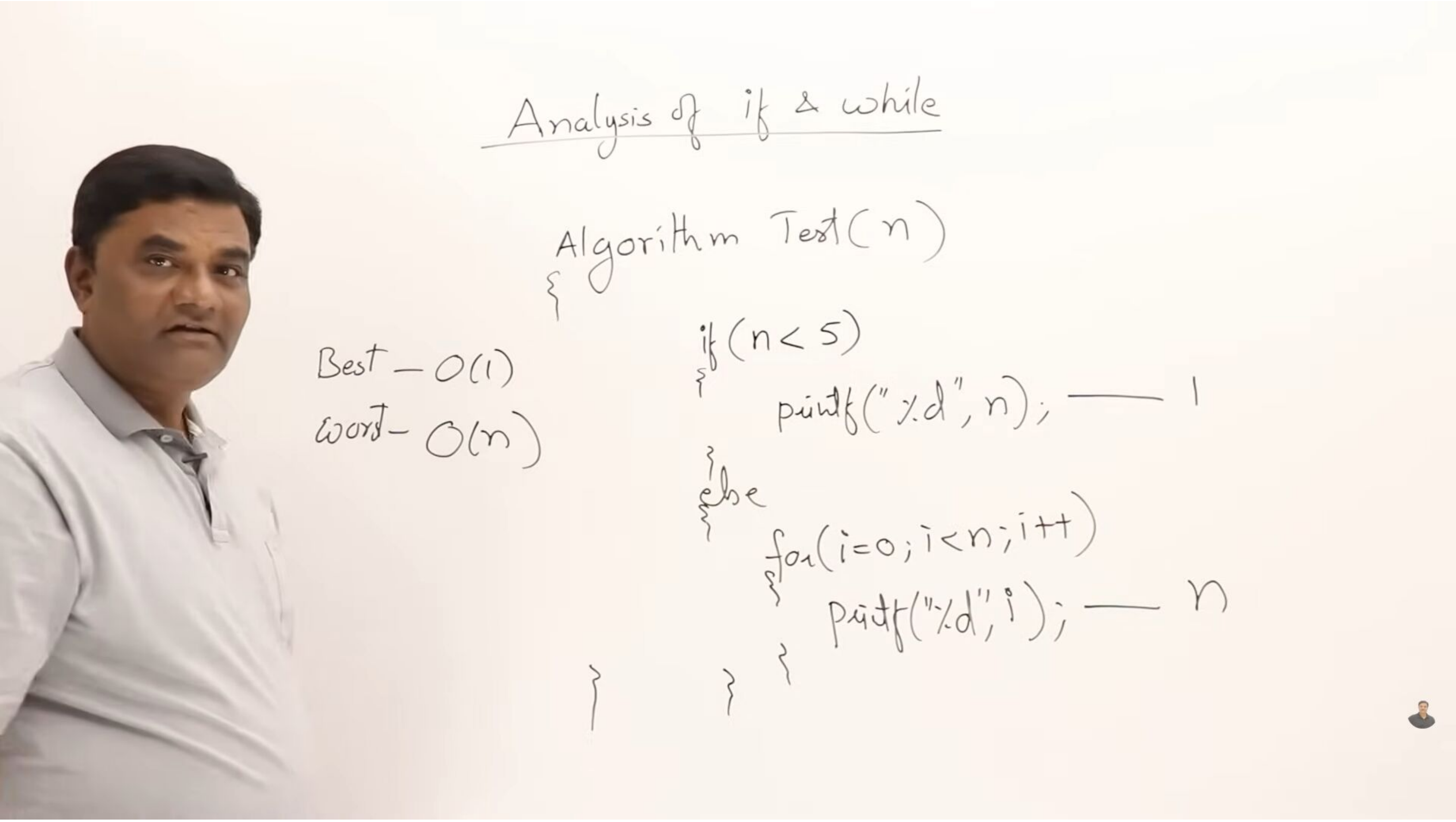

Analysis of if & while
a=1;
while(a<b)
{
stmt;
a=a*2;
}
for(i=1; i<n; i=i*2)
{
stmt;
}
a
1
1×2=2
2×2=2^2
2^2×2=2^3
2^k
Terminate
a ≥ b
∵ a = 2^k
2^k ≥ b
2^k = b
k = log_2 b
O(log n)


Analysis of if & while
i=n;
while(i>1)
{
stmt;
i=i/2;
}
for(i=n; i>1; i=i/2)
{
stmt;
}

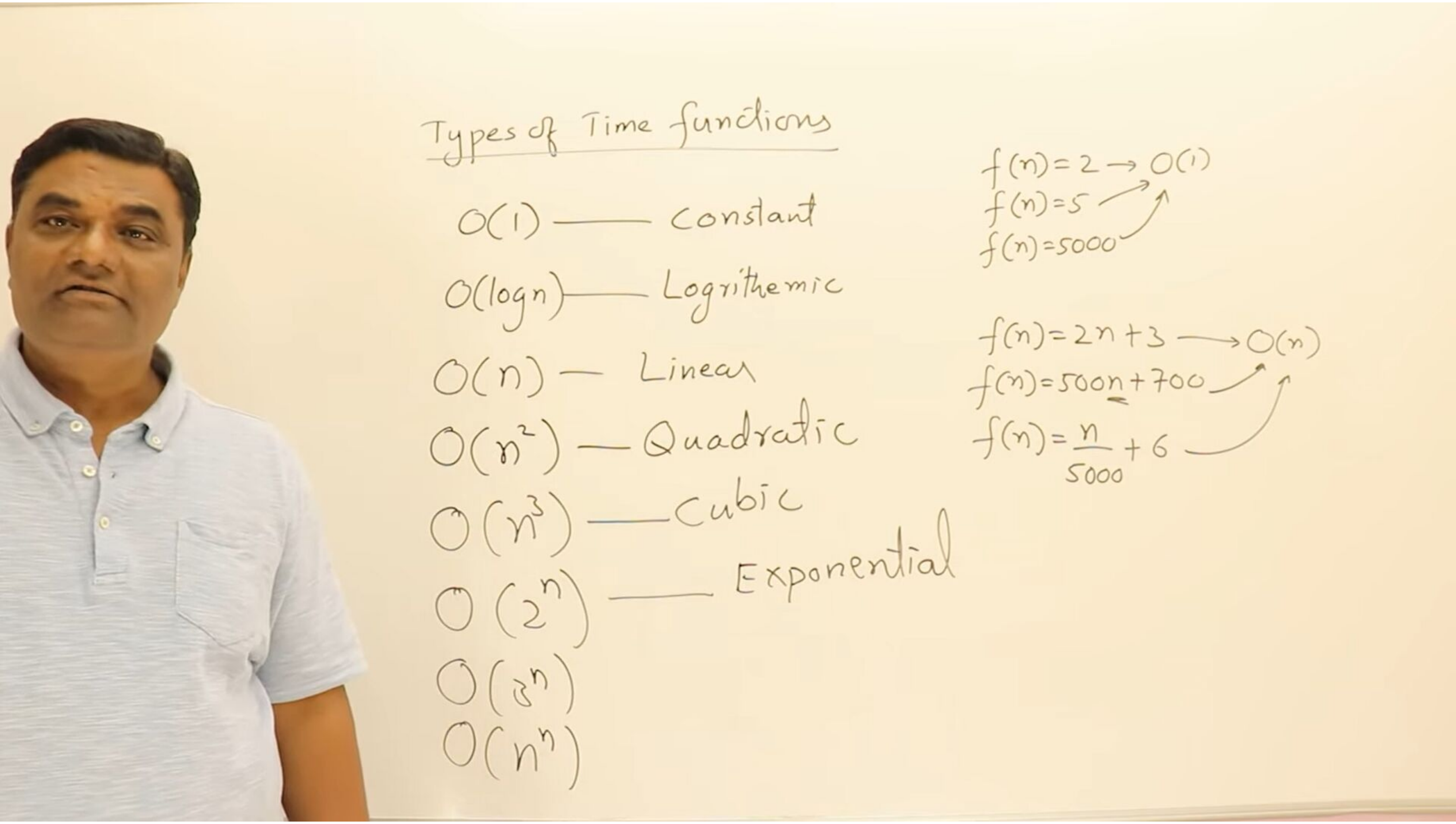

Analysis of if & while
i=1;
k=1;
while(k<n)
{
stmt;
k=k+i,
i++;
}
O(√n)
for(k=1, i=1; k<n; i++)
{
stmt;
k=k+i;
}

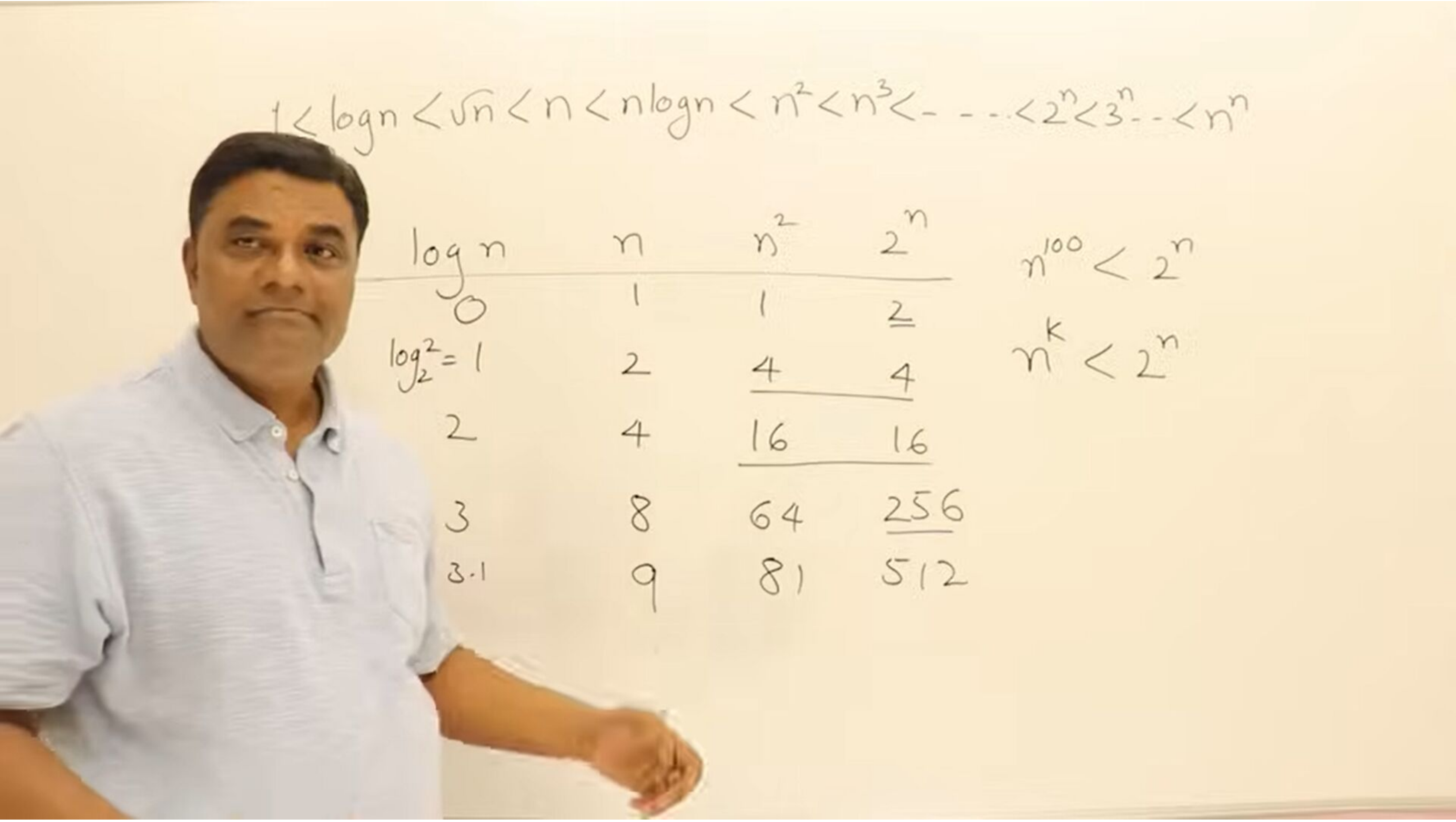

Analysis of if & while
while(m!=n)
{
if(m>n)
m=m-n;
else
n=n-m;
}
m=16 n=2
14 2
12 2
10 2
8 2
6 2
4 2
2 2
16/2
n/2
min O(1)
O(n)

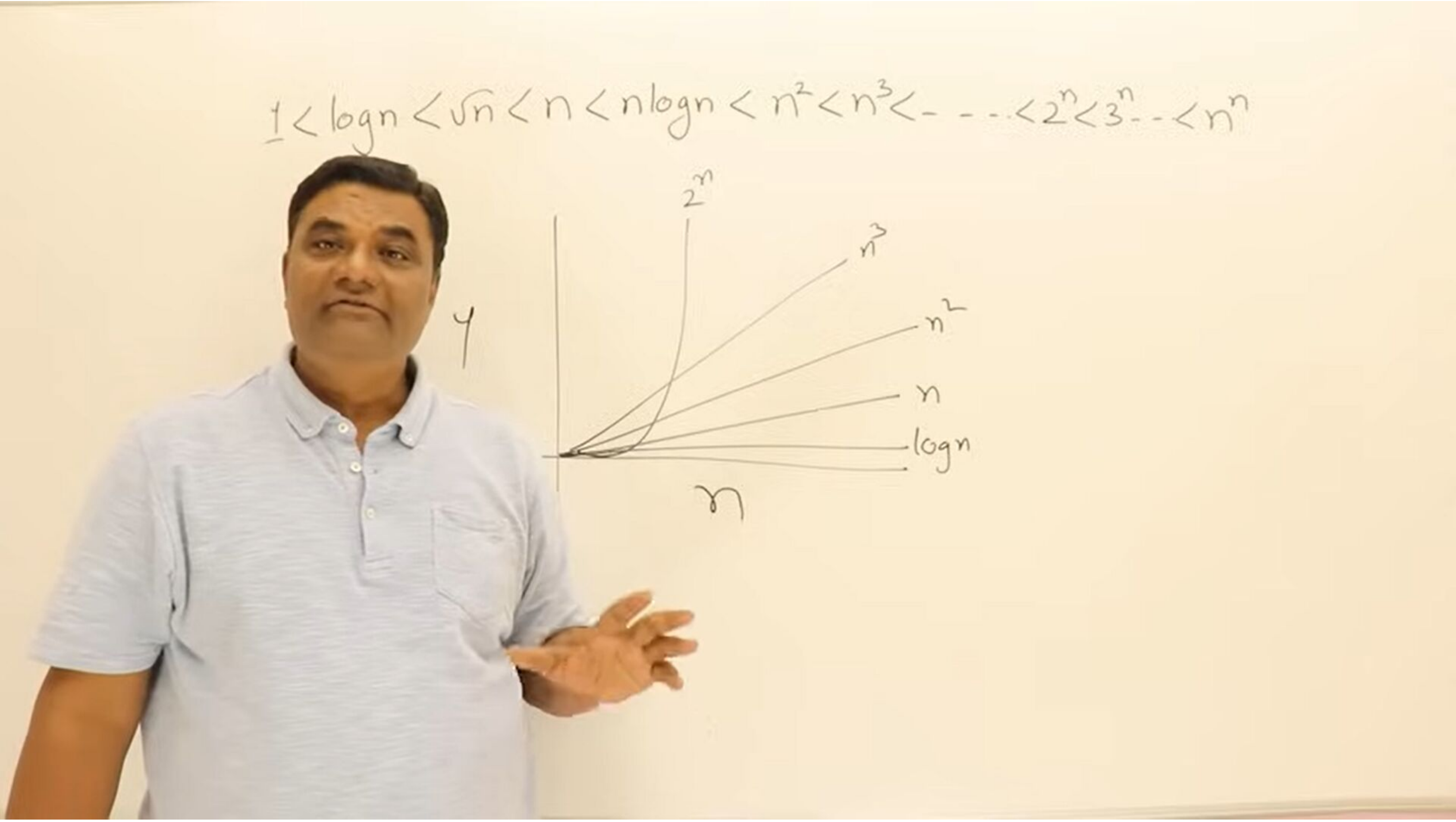

Analysis of if & while
for(; m!=n;)
{
if(m>n)
m=m-n;
else
n=n-m;
}
m=16 n=2
14 2
12 2
10 2
8 2
6 2
4 2
2 2
16/2
n/2
min O(1)
O(n)



Analysis of if & while
Algorithm Test(n)
{
if(n<5)
{
printf("%d",n); — 1
}
else
{
for(i=0;i<n;i++)
{
printf("%d",i); — n
}
}
}
Best – O(1)
Worst – O(n)


Types of Time functions
O(1) — constant
O(logn) — Logrithemic
O(n) — Linear
$O(n^2)$ — Quadratic
$O(n^3)$ — cubic
$O(2^n)$ — Exponential
$O(3^n)$
$O(n^n)$
f(n) = 2 → O(1)
f(n) = 5
f(n) = 5000
f(n) = 2n + 3 → O(n)
f(n) = 500n + 700
$f(n) = \frac{n}{5000} + 6$


$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \cdots < 2^n < 3^n \cdots < n^n$
$\log n$ n n^2 2^n
0 1 1 2
$\log_2 2 = 1$ 2 4 4
2 4 16 16
3 8 64 256
3.1 9 81 512
$n^{100} < 2^n$
$n^k < 2^n$


$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \dots < 2^n < 3^n \dots < n^n$
2^n
n^3
n^2
n
$\log n$
y
n